SkyBook (MERN): three sample technical reports, marked

A high-distinction report, an average report and a failing report for the same project, each followed by a numbered marker's commentary that explains why it meets, partly meets or fails the CA brief and the Part 2 rubric.

Project	**SkyBook (MERN) flight booking**
Brief	**B9IS130 CA 1, Part 2: Technical Report (20 marks)**
Reports	**A: about 4,881 words · B: about 2,059 · C: about 636**
Indicative marks	**A about 88% · B about 55% · C about 12%**

How to use this document

The CA brief asks for a technical report of at least 2,000 words that critically discusses the research process, system architecture, technology choices, security implementation, testing strategy and deployment, with Harvard referencing. This document shows **three versions of that report for the same project** (the SkyBook MERN flight-booking sample), so students and markers can see what the grade bands look like in practice. Each report is followed by **numbered marker comments**. In the report text, small coloured tags (A4 green, B3 amber, C2 red) point to the comment that explains the mark, and each comment names the rubric row and the brief element it relates to.

> **These are teaching exemplars, not templates**
>
> Do not submit or adapt these reports. They describe the sample project, not a student's own work, and copying them would breach the academic-integrity rules in the brief. Use them to see *the kind of reasoning, evidence and referencing* that earns marks, then write about your own project, your own decisions and your own measured results.

> **What is real and what is a placeholder**
>
> Everything the reports say about the system comes from the actual SkyBook (MERN) code: the architecture, data model, security settings, test names and the counts of 14 server and 28 client tests. Things that only a real student can measure appear as [bracketed placeholders]: the student's name, the deployed and repository URLs, the Lighthouse score, the concurrency-test output and the deployment verification. They are not invented.
>
> **References in Report A are real, dated and current as of 4 November 2026.** Standards and regulations are cited at the edition in force on that date (for example OWASP Top 10:2025, ASVS 5.0.0, NIST SP 800-63B-4 and the European Accessibility Act). The access dates in the list show when this exemplar was prepared: a student must record the dates on which *they* read each source. Web addresses and page titles change, so tutors should still re-check every link before reuse.

Marks shown are indicative

The brief gives the Part 2 rubric in bands (Fail, Pass, Merit, Distinction, High Distinction) with three rows but gives no weights for the rows. The marks here assume the three rows count equally, giving a mark out of 20. Your institution's moderation will decide real marks.

What the brief requires of the report

The Part 2 rubric rows

Rubric row	Fail (below 40%)	Pass (40–49%)	Merit (50–59%)	Distinction (60–69%)	High Distinction (70%+)
Research & Analysis	Limited or no research	Basic research evident	Adequate analysis and justification	Strong research and critical evaluation	Excellent critical analysis with extensive supporting evidence
Technical Discussion	Poor explanations	Basic technical discussion	Clear explanations of implementation	Strong technical depth and justification	Advanced technical insight and critical evaluation
Structure & Presentation	Poorly structured with major errors	Basic organisation	Clear and readable	Well structured and professional	Exceptionally professional and academically rigorous

The brief's requirements, checked against all three reports

Brief requirement	A	Why	B	Why	C	Why
Minimum 2,000 words	Meets	About 4,881 words of prose, which is ample.	Just	About 2,059 words: it meets the number, but much of it is descriptive.	Fails	About 636 words: far below the minimum.
Critical discussion of the research process	Meets	Three research strands, spikes, sources; each linked to a decision (A4–A8).	Partly	One weak paragraph; no sources that support it (B3).	Fails	Says little research was done (C5).
System architecture	Meets	Figure, REST constraints, layering, data-model decision with alternatives (A9–A11).	Partly	Accurate description, no figure or trade-offs (B6–B7).	Fails	Colours and layout only (C9).
Technology choices	Meets	Alternatives, reasons, trade-offs, cited (A12).	Partly	"Popular", "easy"; unsupported claim (B5).	Fails	"The lecturer said to" (C4).

Brief requirement	A	Why	B	Why	C	Why
Security implementation	Meets	Threat-to-control table, OWASP mapping, cookie trade-off, residual risk (A18–A20).	Partly	Real controls, but with errors and overclaims (B9–B10).	Fails	“Cannot be hacked” (C6).
Testing strategy	Meets	Strategy, accurate counts, defect log, limits stated (A21–A23).	Partly	Accurate list, no strategy or limits (B11–B12).	Fails	No tests written (C7).
Deployment process	Meets	Configuration, CI, verification, gap analysis (A24–A25).	Partly	Correct steps only (B13).	Fails	No URL, no process (C8).
Harvard in-text citations and full reference list	Meets	36 current sources (28 dated), every citation matches an entry and vice versa (A29).	Fails	Five incomplete entries; citations and list do not match; cites the superseded 2021 OWASP list (B10, B15).	Fails	None at all (C12).
Professional structure and presentation	Meets	Numbered sections, captions, appendices (A30).	Partly	Clear headings, long paragraphs, no figures or captions (B15 area).	Fails	No headings, informal, no figures (C12).
Acknowledging AI, external code and tutorials; understanding it	Meets	Specific declaration (A28).	Fails	Tutorial cited without date; AI use and external code not declared (B1).	Fails	Undeclared external code the writer does not understand (C2).
Working GitHub and deployed URLs	Meets	Both present in the title block (A1).	Meets	Both present (B1).	Fails	Missing (C1, C8).
Accuracy of claims	Meets	Claims are bounded by evidence (A15, A23, A26).	Partly	Overclaims and wrong terms (B9, B10, B14).	Fails	Describes features that do not exist (C3).

Report A: High Distinction (70% and above)

> **Why this report earns 70% or more, in one paragraph**
>
> It does what the three rubric rows describe at their top band. For **Research & Analysis** it treats research as a process, bases decisions on cited evidence, evaluates the project against measurable objectives and admits what its own evidence cannot show. For **Technical Discussion** it explains *why* each design works, compares alternatives with stated trade-offs, quotes real code and names its limits. For **Structure & Presentation** it is numbered, captioned, consistent and referenced in full Harvard style. It covers every item the brief lists, in the order a reader needs them, and it is honest. At about 4,881 words it is far longer than the minimum, but **length is not what earns the marks**: Report B is long enough too.

SkyBook: a secure, accessible flight-booking system built with the MERN stack

Technical report for B9IS130 Web Development for Information Systems, Continuous Assessment 1 (Part 2)

Student: [name and student number] · Deployed application: [public URL] · GitHub repository: [repository URL] · Date: [date] · Word count: about 4,881 (excluding tables, code, figures, references and appendices) · Use of AI tools and external code is declared in Section 8.6. A1

Abstract

Flight booking is a deceptively demanding problem for an information system. It combines a scarce shared resource (seats), money, personal data and time-based business rules. This report documents SkyBook, a flight search and booking application for a small regional airline, built with MongoDB Atlas, Express, React and Node.js and deployed to a public cloud platform. The central design problem was to guarantee that two customers can never be sold the same seat, even when they click at the same instant. I solved it by embedding the seat map in each flight document and reserving every requested seat with a single conditional update, an approach that I justify against a transaction-based alternative. Other contributions are server-side fare calculation, a keyboard-accessible seat map, a layered security design mapped to the OWASP Top 10:2025, and an external API integration that fails gracefully. The system is covered by 42 automated tests (14 server and 28 client), and the report critically evaluates what those tests do and do not prove. It closes with an honest account of limitations, legal and ethical considerations, and prioritised future work. A2

1. Introduction

1.1 Problem and context

A booking system is a textbook case of concurrent access to shared state. If two requests read the same free seat and both write a booking, one customer arrives at the airport without a seat. Kleppmann and Riccomini (2026, ‘Transactions’ chapter) describes how anomalies such as lost

updates and write skew arise when concurrent transactions are not isolated, and why the choice of database guarantee matters. The same domain adds a financial dimension (fares and refunds that must be exact), a privacy dimension (passenger names are personal data) and an accessibility dimension (a seat map is a visual interface that must still be usable without a mouse or without sight).

The project brief asks for a full-stack information system that demonstrates requirements analysis, front-end and back-end integration, database management, API integration, security, software engineering practice and cloud deployment. I chose a flight-booking domain because it forces explicit decisions about each of those areas rather than allowing them to be satisfied superficially.

1.2 Aim and objectives

The aim is to design, build, test, secure and deploy a flight search and booking system that is correct under concurrent use, accessible, and maintainable. The aim is broken into five measurable objectives:

1. **O1 Correctness:** no seat is ever sold twice, and a multi-seat booking is all-or-nothing.
2. **O2 Security:** the system implements authentication, authorisation, input validation and protection against the OWASP Top 10 risks that are relevant to its design.
3. **O3 Usability and accessibility:** every workflow is operable by keyboard and meets the relevant WCAG 2.2 success criteria that I can test.
4. **O4 Integration:** the system consumes one external web service and continues to work when that service fails.
5. **O5 Engineering quality:** the code is layered, tested automatically, version controlled and deployed to a public platform from a repeatable build.

A3

1.3 Scope and assumptions

Payments are simulated and flights are generated sample data, because real airline distribution APIs and payment providers require commercial accounts and secret keys that cannot be exposed in a coursework repository. Only single-leg flights are supported. These boundaries are stated so that the evaluation in Section 8 can judge the system against what it set out to do, not against a full airline platform.

1.4 Structure of the report

Section 2 presents the research and requirements analysis. Section 3 explains the architecture and technology choices. Section 4 describes the key implementation decisions. Sections 5, 6 and 7 cover security, testing and deployment. Section 8 critically evaluates the work, and Section 9 concludes.

2. Research and requirements analysis

2.1 Research approach

The research ran in three strands, each producing evidence that fed into a design decision. First, a *user and domain* strand drew on usability and accessibility standards: Nielsen’s usability heuristics (Nielsen, 1994) and the WCAG 2.2 success criteria (W3C, 2023) shaped the interface requirements.

Second, a *technical literature* strand examined transactions and concurrency (Gray, 1981; Haerder and Reuter, 1983; Kleppmann and Riccomini, 2026, 'Transactions' chapter), web security (the current OWASP Top 10:2025 and Application Security Verification Standard 5.0.0) (OWASP Foundation, 2025b; OWASP Foundation, 2025a) and software architecture (Bass, Clements and Kazman, 2021; Fowler, 2002). Third, a *prototyping* strand used three one-day spikes in week 1 to remove the biggest technical unknowns: (i) a vertical slice from a React page through Express to MongoDB Atlas, following the structure of the official MERN tutorial (MongoDB, Inc., n.d.) but replacing its unauthenticated, unvalidated design; (ii) a test of whether a single MongoDB update could reserve several array elements atomically; and (iii) a call to the REST Countries API (REST Countries, n.d.) to learn its response shape and failure behaviour. The second spike determined the data model described in Section 3.3. A4

2.2 Stakeholders and user needs

Table 1 lists the stakeholders, their needs and the design consequence of each. The "group" stakeholder is easy to overlook and drives the all-or-nothing requirement: a family of three must not end up with two seats. The assistive-technology stakeholder follows from WCAG 2.2 (W3C, 2023) and from a legal duty: the European Accessibility Act, applicable since 28 June 2025, extends accessibility requirements to the websites, mobile applications and electronic ticketing of air passenger transport services (European Parliament and Council of the European Union, 2019). Exemptions exist (for example for microenterprises), so a real airline would need legal advice on its own position; for this project I treated the requirements as binding.

Stakeholder	Need	Design consequence
Traveller	Find a flight quickly, choose seats, understand the price and refund terms before committing	Clear results, seat map, price estimate, refund policy shown on the bookings page (heuristics 1 and 5; (Nielsen, 1994))
Group or family	Sit together; either all seats are confirmed or none	Single atomic multi-seat reservation (O1)
Traveller using assistive technology	Operate the seat map without a mouse or sight	Seats are buttons with descriptive labels; status is not conveyed by colour alone (WCAG 1.4.1, 2.1.1, 4.1.2; European Accessibility Act)
Airline operations (administrator)	Add flights; see bookings and revenue	Role-based administrator area; role cannot be self-assigned
Data subjects (passengers)	Minimal, protected processing of personal data	Store names only (no passport data); hashed passwords; data minimisation (European Parliament and Council of the European Union, 2016)

Table 1. Stakeholder analysis.

A5

Regulation also frames the refund rules. Regulation (EC) No 261/2004 (European Parliament and Council of the European Union, 2004) governs passenger rights when an airline cancels or delays a flight. SkyBook's refund policy concerns the different case of a *passenger-initiated* cancellation, so the tiers I implemented (100% more than 72 hours before departure, 50% from 24 to 72 hours, none inside 24 hours) are an illustrative commercial policy, not a legal template. Making the policy a single pure function (Section 4.3) means an airline can change it in one place.

2.3 Functional requirements

ID	Requirement	Priority	Acceptance criterion
FR1	Search flights by origin, destination and date without logging in	Must	Searching DUB→LHR for tomorrow returns the seeded flights with free-seat counts; unknown airports or past dates are rejected
FR2	View a flight with its seat map and destination facts	Must	Seat map shows 48 seats with booked seats disabled; destination card appears or a polite message replaces it
FR3	Register, log in and log out	Must	Weak passwords are rejected; the session survives a refresh; logout clears it
FR4	Book 1 to 4 seats with passenger names; fare calculated by the server	Must	Total equals base fare per passenger plus legroom surcharge; a tampered price in the request has no effect
FR5	A seat can never be sold twice; a multi-seat booking is all or nothing	Must	Two simultaneous requests for one seat: exactly one succeeds; a request mixing free and taken seats changes nothing
FR6	A secure booking reference is shown to the customer	Must	Six characters from an unambiguous alphabet, generated with a cryptographic random source
FR7	View own bookings and cancel with a time-based refund	Must	Refund is 100%, 50% or 0% according to the policy; a second cancel is refused
FR8	Administrator can create flights and view statistics	Should	Only users with the admin role can reach the endpoints (403 otherwise)
FR9	Customers see only their own bookings	Must	Another user’s booking reference returns 404
FR10	Payments, seat changes, baggage, multi-city trips	Won’t	Documented as future work

Table 2. Functional requirements with acceptance criteria and MoSCoW priority.

A6

2.4 Non-functional requirements

ID	Quality	Measurable target	Verification
NFR1	Consistency	No double-booking under simultaneous requests	Mocked service tests plus a two-browser test and a scripted burst of simultaneous requests
NFR2	Security	Controls mapped to the relevant OWASP Top 10:2025 categories; no secrets in version control	Attack tests; repository review
NFR3	Accuracy	All money in whole cents; refund boundaries correct at exactly 24 h and 72 h	Unit tests at the boundaries
NFR4	Accessibility	Keyboard-operable seat map; visible focus; contrast meets WCAG 2.2 AA	Component tests by role and name; keyboard walk-through; automated audit [Lighthouse accessibility score]
NFR5	Responsiveness	Usable from 320 px; no sideways page scroll	Browser device toolbar

ID	Quality	Measurable target	Verification
NFR6	Reliability	Booking still works if the country service is down	Failure test with the service blocked
NFR7	Performance	Search served by an index; external facts cached for 24 hours	Index inspection; cache unit test
NFR8	Maintainability	Layered code; tests run on every push	Repository structure; CI run

Table 3. Non-functional requirements with measurable targets.

A7

2.5 Feasibility, risk and planning

The project used an iterative approach in which each week produced a working vertical slice rather than a finished layer (Sommerville, 2016). Requirements were prioritised with MoSCoW so that scope could shrink without losing the core booking path. Table 4 summarises the main risks.

Risk	Likelihood / impact	Mitigation
Concurrency bug appears only under load	Medium / High	Design the atomic update first (spike ii); test it with simulated simultaneous requests
Third-party API changes or is down	Medium / Medium	Server-side adapter, timeout, cache and graceful fallback; booking never depends on it
Free hosting sleeps or limits resources	High / Low	Wake the site before demonstrations; document cold-start behaviour
Personal-data obligations	Low / High	Collect the minimum; no payment or passport data; hashed passwords (European Parliament and Council of the European Union, 2016)
Scope creep (baggage, connections)	High / Medium	MoSCoW “Won’t” list; single-leg flights only

Table 4. Risk register.

A8

3. Architecture and technology choices

3.1 Overall architecture

SkyBook is a three-tier system (Figure 1): a React single-page client, an Express API server and a MongoDB Atlas database, plus one external web service. The API follows the REST constraints of statelessness and a uniform interface (Fielding, 2000): each request carries its own credentials (a signed token in a cookie) and the server keeps no session memory, so any instance can answer any request. Architectural decisions were made by explicit trade-off between quality attributes (Bass, Clements and Kazman, 2021). In production, one Express service serves both the API and the built client. That choice trades some deployment flexibility (the front end cannot be scaled or cached separately) for same-origin requests, which remove cross-origin configuration and allow a strict `SameSite` cookie (Section 5.3).

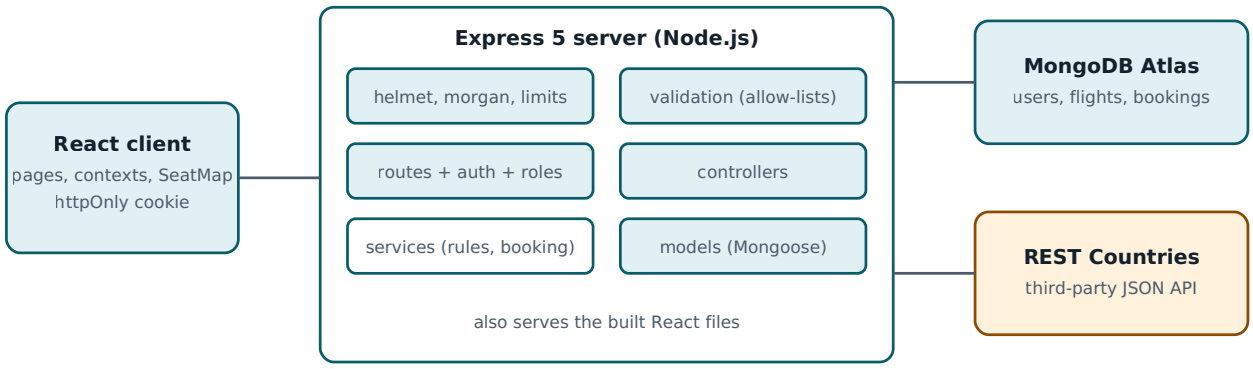


Figure 1. SkyBook architecture. In development the client runs on its own port and a proxy forwards `/api`; in production Express serves both.

A9

3.2 Layering on the server

The server separates concerns into routes (which URL, which checks), controllers (translate HTTP to a function call), services (business rules) and models (data shape and constraints). This is the layered structure described by Fowler (2002) and supports the readability and small-function principles in Martin (2008). The practical benefit is testability: the fare, seat and refund rules are pure functions with no database or network, and the booking service can be tested with mocked models, so the most important logic is checked without a running database. A10

3.3 Data model and the key design decision

The database has three collections: users, flights and bookings. Users have a unique email and a role. Bookings reference a user and a flight and store the passengers, the total, the status and any refund. The significant decision concerns seats. MongoDB guarantees that a write to a single document is atomic (Bradshaw, Brazil and Chodorow, 2019), so if a flight’s seats are embedded in the flight document, reserving several seats can be one write. The alternative is a separate seat collection, which keeps documents small but needs multi-document transactions to make a multi-seat reservation all-or-nothing. Table 5 compares them.

Criterion	Embedded seats (chosen)	Separate seat collection with transactions
Atomic multi-seat reservation	One conditional update on one document	Requires a transaction across many documents
Complexity	Low: no transaction code or retry logic	Higher: sessions, transaction retries, error cases
Document size	48 seats is a few kilobytes; far below the 16 MB document limit	Small documents
Contention	All bookings on one flight touch one document, so a very popular flight serialises writes	Writes spread across seat documents
Query for “seats left”	Computed inside the database with an aggregation	Count query

Criterion	Embedded seats (chosen)	Separate seat collection with transactions
Scales to large aircraft or many simultaneous buyers	Weaker	Stronger

Table 5. Embedded versus referenced seats.

For a 48-seat aircraft the embedded design is simpler and sufficient, and it removes a whole class of failure (a half-completed transaction). The cost is contention on very popular flights, which I record as a limitation and a trigger for redesign. The decision follows the document-modelling guidance in the MongoDB literature that data accessed together should usually be stored together (Bradshaw, Brazil and Chodorow, 2019), and it uses the atomicity, consistency, isolation and durability vocabulary from the transaction literature (Gray, 1981; Haerder and Reuter, 1983). I explicitly avoided assuming that transactions are always the right tool: a simpler guarantee that fits the shape of the data is easier to reason about and to test. A11

3.4 Technology selection

Decision	Chosen	Alternatives considered	Reason and trade-off
Stack	MERN (JavaScript throughout)	MEAN; non-JavaScript stacks	The brief prefers JavaScript stacks; one language lowers context switching. React's smaller core suits a project where I wanted to choose routing and forms deliberately (The React Team, 2024)
Server framework	Express 5	Fastify, Koa	Mature, widely documented middleware model; Express 5 forwards errors from asynchronous handlers to the error handler (Todd, 2024)
Data access	Mongoose	Native driver	Schemas, indexes and validation in one place; modest overhead (Mongoose, n.d.)
Validation	express-validator	Zod, Joi	Handles arrays and nested fields (`passengers.*.seat`) and custom rules (express-validator, n.d.)
Authentication	Signed token in an httpOnly cookie	Server-side sessions; token in browser storage	See Section 5.3 for the trade-off
Build tool	Vite	Webpack, Create React App	Fast development server and simple configuration
Hosting	Render and Atlas free tiers	Other platforms	No cost; deploys from the repository (Render, n.d.); cold starts and a wide network allow-list are accepted trade-offs (Section 7)

Table 6. Technology choices, alternatives and justification.

A12

3.5 External API integration

The flight page shows facts about the destination country (name, capital, languages, currency) from the REST Countries API (REST Countries, n.d.). The call is made by the *server*, not the browser. That keeps third-party details out of the client, avoids cross-origin restrictions, and lets the server control timeouts and caching. The integration has four defensive layers: a request timeout of six seconds; an *adapter* that converts the provider's response (an object or an array, with optional fields) into a

small stable shape; a 24-hour in-memory cache, because country facts rarely change and a free service deserves polite use; and a typed error that the controller turns into a 502 reply. The page treats the destination card as optional, so a failure removes the card, not the ability to book (NFR6). A13

3.6 Front-end design and accessibility

The client has pages for search, flight and seats, bookings, administration and login, with shared state (the logged-in user) in a React context and a single fetch wrapper that sends the cookie, parses JSON and converts errors into readable messages. The accessibility design is the most distinctive part. The seat map is a grid of real `button` elements, not clickable images, which gives keyboard operation (WCAG 2.1.1) and an accessible name and state through `aria-label` and `aria-pressed` (4.1.2) (W3C, 2023). For an airline serving EU customers these requirements are also a legal matter (European Parliament and Council of the European Union, 2019). Taken seats are disabled *and* struck through, so information is not conveyed by colour alone (1.4.1). Form fields have visible labels programmatically associated with their inputs (1.3.1), errors are announced with `role="alert"` (3.3.1), and focus outlines are always visible (2.4.7). On narrow screens the seat map scrolls inside its own container so the page never scrolls sideways. Nielsen's heuristics (Nielsen, 1994) shaped the interaction: the number of seats left (visibility of system status), a cap of four passengers and disabled taken seats (error prevention), and popular-route shortcuts (recognition rather than recall). A14

4. Implementation highlights

This section explains four decisions that carry the most risk or learning value. The complete code is in the repository [repository URL].

4.1 Atomic multi-seat reservation

The booking service first loads the flight to check that it exists, has not departed and that every requested seat exists. It then performs the reservation as one update:

server/src/services/bookingService.js (excerpt)

```
  const result = await Flight.updateOne(
    {
      _id: flightId,
      $and: seatNumbers.map((n) => ({ seats: { $elemMatch: { number: n, booked: false } } })),
    },
    { $set: { "seats.$[s].booked": true, "seats.$[s].bookingRef": ref } },
    { arrayFilters: [{ "s.number": { $in: seatNumbers } }] }
  );
  if (result.modifiedCount !== 1) throw new BookingError("Sorry, one or more of those seats was just
taken. Please choose again.");
```

The filter is a conjunction (`$and`) with one `$elemMatch` per requested seat, so it matches only if *every* requested seat is currently free. The update uses an `arrayFilters` entry to set only the matching array elements, tagging them with a freshly generated booking reference. Because the whole operation is a single-document write, it is atomic (Bradshaw, Brazil and Chodorow, 2019): if two requests race for the same seat, the database applies one first and the second filter no longer matches, so `modifiedCount` is 0 and the service throws a conflict error that becomes a calm 409 message. The same mechanism makes a request that mixes a free seat and a taken seat change

nothing, which is the all-or-nothing behaviour (FR5). The price is not taken from the request: it is calculated from the stored fare and the chosen seats. A15

4.2 Compensation and idempotent cancellation

Reserving seats and saving a booking are two writes, and the second can fail after the first has succeeded. Rather than leave seats locked with no booking, the service releases them in a compensating step:

server/src/services/bookingService.js (excerpt)

```
  try {
    return await Booking.create({
      ref,
      user: userId,
      flight: flightId,
      passengers,
      totalCents: calculateFare(flight.priceCents, seatNumbers), // price is calculated here, never
sent by the browser
    });
  } catch (err) {
    await releaseSeats(flightId, ref); // compensating action: do not leave seats locked without a
booking
    throw err;
  }
```

This is a small instance of the saga idea of undoing earlier steps with compensating actions instead of relying on one large transaction (Garcia-Molina and Salem, 1987). It is not equivalent to a transaction: if the server crashes between the two writes the compensation never runs, which I record as a limitation. Cancellation uses a related technique. The status change is a conditional update that applies only while the booking is still `confirmed`, so a double click or a repeated request cannot refund twice (idempotence). Seats are released by matching their booking reference. A16

4.3 Business rules as pure functions with injectable time

The refund policy and the fare calculation are small functions with no database or network access. The refund function receives the current time as a parameter instead of reading the clock:

server/src/services/cancellationPolicy.js (excerpt)

```
export function refundPercent(departureTime, now = new Date()) {
  const hoursLeft = (new Date(departureTime).getTime() - now.getTime()) / HOUR;
  if (hoursLeft <= 0) return null; // already departed: cannot cancel
  if (hoursLeft > 72) return 100;
  if (hoursLeft >= 24) return 50;
  return 0;
}
```

Passing `now` in makes the boundaries testable: the tests check exactly 72 hours (which falls in the 50% band), exactly 24 hours (also 50%) and one hour either side. Money is held as whole cents to avoid floating-point error. These choices follow the principle that small, single-purpose functions with explicit inputs are easier to understand and test (Martin, 2008), and they keep the business policy in one place. A17

4.4 A client that recovers from a lost race

A race between two customers is rare but certain to happen eventually, so the client treats it as a normal path. When the booking request fails, the page shows the server's message, clears the chosen seats and reloads the flight:

client/src/pages/FlightPage.jsx (excerpt)

```
    } catch (err) {
      setError(err.message);
      setSelected([]); // seats may have been taken: reload the map and let the user choose again
      load();
```

Reloading changes the flight held in React state, so the seat map is redrawn with the seat now disabled, and the customer can choose again. The estimated total on the page is labelled "estimated" because the server owns the real price.

5. Security

5.1 Approach

I used a lightweight threat-modelling pass in the spirit of STRIDE (Shostack, 2014): for each asset (accounts, bookings, seat inventory, secrets) I asked who could spoof, tamper, disclose, deny service or escalate privilege, and then chose a control. Table 7 maps the main threats to controls and to the categories of the OWASP Top 10:2025 (OWASP Foundation, 2025b), the edition published in November 2025, and where useful to the Application Security Verification Standard (OWASP Foundation, 2025a). The 2025 list re-orders and renames several categories (supply-chain failures and the mishandling of exceptional conditions are now categories in their own right), so reports that quote the 2021 numbering need care.

Threat	OWASP Top 10:2025 category	Control in SkyBook
A customer edits the request to pay less	A06 Insecure Design	Fares are calculated on the server from stored prices and the chosen seats; the request carries no price
Reading or cancelling another user's booking	A01 Broken Access Control	Every booking query includes the owner; a foreign reference returns 404, which does not reveal that it exists
Ordinary user reaches administrator functions	A01	Role middleware on the whole admin router (403); the role cannot be set through the API, only by a server-side script
Cross-site request forgery and a weak cookie policy	A01 (lists CSRF and improper SameSite settings among its weaknesses)	`SameSite=Strict` cookie, JSON-only API, same-origin deployment
Redirect to an attacker's site after login	A01 (lists open redirect)	The client only follows a "return to" address if it is a path inside the app
Stolen or guessed password; weak storage	A07 Authentication Failures; A04 Cryptographic Failures	bcrypt hashing at cost 12; one generic login error; rate limit of 20 attempts per 15 minutes. Password *policy* is a known gap (Section 5.2)

Threat	OWASP Top 10:2025 category	Control in SkyBook
Query-operator injection (NoSQL) and script injection	A05 Injection	Type checks reject objects where text is expected; airports and seats are allow-listed; React escapes output; Content-Security-Policy from Helmet
Extra fields smuggled into a request (mass assignment)	A06 Insecure Design	Validation keeps only known fields; passenger objects are rebuilt from `name` and `seat`
Request flooding, oversized bodies, slow third party	A06	Rate limits on login and booking; 10 kb JSON limit; timeout on the external API
Internal details leaking through errors; unhandled failures	A10 Mishandling of Exceptional Conditions; A02	Central error handler returns generic text in production; asynchronous errors are forwarded to it; typed errors for known cases
Leaked secrets; unsafe defaults	A02 Security Misconfiguration	Settings from environment variables; `.env` excluded from version control; start-up check refuses a short secret
Vulnerable or compromised dependencies	A03 Software Supply Chain Failures	Lock files committed; dependency audit before submission; clean installs in CI; few dependencies
Attacks go unnoticed	A09 Security Logging and Alerting Failures	Request log only: *no alerting*, recorded as a residual risk

Table 7. Threat-to-control mapping to OWASP Top 10:2025.

A18

5.2 Authentication and password storage

Passwords are never stored. The server stores a bcrypt hash with a cost factor of 12. bcrypt was designed to be deliberately slow and to carry a per-password salt so that hardware improvements can be offset by raising the cost (Provos and Mazières, 1999). I used a pure-JavaScript implementation so the project installs on any machine without compiling native code; it is slower than native bindings, which is acceptable at this scale and noted as a trade-off. Login returns the same message whether the email or the password is wrong, so the response does not reveal which emails are registered.

The *policy* for choosing a password is weaker than the storage. SkyBook requires 8 to 72 characters with an upper-case letter, a lower-case letter and a digit. That reflects older convention. The current guidance on digital identity (NIST, 2025) says verifiers must not impose composition rules, requires a minimum of 15 characters where a password is the only authenticator, expects long passwords and passphrases to be accepted (at least 64 characters), and requires candidate passwords to be screened against lists of common or compromised values. The 72-character limit is imposed by bcrypt and is compatible with that range. I therefore treat my own policy as a recorded weakness and list the change as a priority in Section 8.5, rather than presenting composition rules as best practice. A31

5.3 Session design: a deliberate trade-off

After login the server issues a signed JSON Web Token (Jones, Bradley and Sakimura, 2015) with a seven-day expiry and sets it in a cookie marked `httpOnly` (defined in the cookie standard (Barth,

2011)), `SameSite=Strict` and, in production, `Secure`. `SameSite` is specified in the cookie draft that is replacing that standard (Bingler, West and Wilander, 2025), which I therefore cite as work in progress. Current best practice for tokens (Sheffer, Hardt and Jones, 2020) advises explicitly restricting the accepted signing algorithm; the library I use limits a shared secret to the HMAC family by default, and pinning the algorithm explicitly is a small improvement I have recorded. The alternative is to return the token to JavaScript and keep it in browser storage. Storage that scripts can read is exposed to any cross-site scripting flaw, whereas an httpOnly cookie cannot be read by scripts at all; the cookie, however, is attached automatically to requests, which creates cross-site request forgery risk. `SameSite=Strict` addresses that risk for this design, and serving the client and API from one origin makes the strict setting practical. The accepted limitation is that a stateless token cannot be revoked before it expires; fixing that would need short-lived tokens with a refresh mechanism or a server-side session store. **A19**

5.4 Validation and a defect found by testing

Every route that accepts input passes through validation rules that act as an allow-list, consistent with the positive-validation approach of the ASVS (OWASP Foundation, 2025a) and implemented with express-validator (express-validator, n.d.). Testing the booking rules in isolation exposed a subtle problem: the validator’s result keeps whole passenger objects, so an extra key such as `role` would survive validation. The controller therefore rebuilds each passenger from `name` and `seat` only. I record honestly that this fix is covered by manual verification and not yet by a dedicated automated test (Section 6.5 and Section 8.3). **A20**

5.5 Residual risks

The remaining risks are known and accepted for a coursework system: no token revocation; CSRF defence relies on `SameSite` alone rather than on tokens as well; the rate limiter keeps counts in memory, so it resets on restart and is per instance; there is no email verification or password reset; and security events are logged only as ordinary request lines.

6. Testing strategy

6.1 Approach

The strategy follows the test-pyramid idea of many fast, low-level tests and fewer, slower, broader ones (Cohn, 2009; Fowler, 2012), adapted to a coursework budget. The fast base is pure-function and component tests that run in milliseconds; the middle is API tests that exercise routes, validation and security headers without a database; the top is manual and scripted checks against the deployed system. Automated tests run on every push through a workflow (GitHub, n.d.), so a regression is caught within minutes.

6.2 Server tests (14)

The server tests use the Node.js built-in test runner (OpenJS Foundation, n.d.) with a request-testing library. They cover the seat map and fare rules; booking-reference format and uniqueness; the refund policy at its exact boundaries; protected routes returning 401; validation of search input before the database is touched; the airport list, a NoSQL-injection attempt at login and the presence of security headers; the country adapter with both response shapes and missing fields; the cache and failure handling of the external call; and the booking service with mocked models, including the lost race, release of seats when saving fails, rejection of departed flights and unknown seats, and a cancellation that cannot happen twice. Mocks are used for the models because the aim of these

tests is to prove the *logic* (what is sent to the database and what happens on each outcome) quickly and deterministically.

6.3 Client tests (28)

The client tests use Vitest with Testing Library, whose guiding principle is that tests should resemble how software is used (Testing Library, n.d.). They locate controls by role and accessible name, as a user or screen reader would. They cover the fetch wrapper (cookie sent, errors converted, session expiry handled), the login context, route protection, the format helpers, the seat-map component (labels, disabled taken seats, selection, maximum) and the flight page, which is driven like a user: choose seats, type names, book. One test proves that the booking request contains the flight and passengers but no price; another proves that a “seat just taken” reply clears the selection and reloads the seat map. A side effect of testing by role is accessibility evidence: if a control cannot be found by role and name, assistive technology probably cannot find it either.

Layer	Tool	Tests	What it proves
Pure rules and adapters	Node test runner	7 of 14	Fares, refunds, references, adapter behaviour at boundaries
API guard rails	Node test runner + request library	3 of 14	Authentication, validation, injection rejection, headers
Booking service with mocks	Node test runner	4 of 14	Lost race, rollback, departed flight, double cancel
Client units and components	Vitest + Testing Library	28	Accessible UI behaviour, request bodies, recovery path
Manual and scripted	Browser, curl, script	Section 6.4	Security attacks, concurrency, responsiveness

Table 8. Test layers and results (42 automated tests, all passing at submission).

A21

6.4 Manual, security and non-functional testing

Security attacks were run against my own deployment using a cookie jar: no login (401); NoSQL operator objects at login (400); a tampered `totalCents` in a booking (ignored); a second user requesting or cancelling the first user’s booking (404); a normal user calling an administrator endpoint (403); more than 20 login attempts in 15 minutes (429); and a stored script string in a passenger name (rendered as text). The concurrency requirement (NFR1) was tested by two browsers choosing the same seat and by a script sending simultaneous booking requests for one seat; the expected result is that exactly one succeeds. Measured results: [record the number of requests sent, the number that succeeded, and the seat’s final state in the database]. Accessibility was checked by a keyboard-only walk-through of every workflow and an automated audit scoring [Lighthouse accessibility score], followed by inspection at a 390 px viewport.

6.5 Defects found and what they taught

Defect	How found	Fix	Lesson
Extra keys inside passenger objects passed validation	Exercising the validation chain with an unexpected field	Controller rebuilds each passenger from known fields	Validation libraries differ in what they strip; test the chain, not only the happy path
A test of reference uniqueness could fail by chance	Reasoning about the birthday problem on a 32-symbol, six-character space	Reduced the sample to 100 and documented the probability	Randomised tests must have negligible false-failure rates
Link-styled buttons did not look like primary buttons	Looking at screenshots of the running page	Added the more specific selector `a.button.primary`	CSS specificity; inspect the real page, not only the code
A function named `usePopular` looked like a React hook	Code review against hook naming conventions	Renamed to `applyPopular`	Names carry meaning for tools and readers

Table 9. Defect log.

A22

6.6 Limits of the testing

The mocked tests prove that the service *sends* the intended update and handles each outcome; they cannot prove that a real database applies the update atomically. That guarantee comes from the database documentation (Bradshaw, Brazil and Chodorow, 2019) and from the manual concurrency test above, which is therefore the most important piece of evidence for objective O1. There are no automated end-to-end browser tests and no automated integration tests against a temporary database; both are listed as future work. The client tests replace the network with fakes, so they say nothing about real HTTP behaviour. A23

7. Deployment

The application is deployed as a single web service on a public cloud platform (Render, n.d.) at [public URL], with the database on MongoDB Atlas. The build command installs dependencies and builds the client; the start command runs the server, which serves the API and the built client from one address. Configuration is supplied through environment variables (the database connection string, the token secret and the production flag), never through files in the repository. A health endpoint lets the platform check that the service is alive. The same build and test commands run in a continuous-integration workflow (GitHub, n.d.) on every push, so what is deployed has passed the automated tests. A24

Verification followed a checklist: the health endpoint responds; the home page is fully styled; a user can register, search, choose seats, book and cancel; refreshing a deep link such as the bookings page works because the server returns the client for non-API paths; the data appears in Atlas; and the site works on a phone over mobile data. Results: [date, platform region and outcome of the checklist].

Table 10 states the gap between this deployment and a production system.

Area	Current state	What production would need
Database network access	Atlas allows any address (0.0.0.0/0) because the free host’s outgoing addresses change; protected by a long random password and a restricted database user	Static outbound addresses or private networking, so the allow-list can be narrow
Availability	Free instance sleeps after inactivity; the first request can take about a minute	Always-on instance, health monitoring, alerting
State	Rate-limit counters and the country cache live in process memory	A shared store such as Redis for several instances
Backups and recovery	Provider defaults only	Tested backups and a recovery procedure
Observability	Request log lines	Structured logs, error tracking and security-event alerts

Table 10. Deployment gap analysis.

A25

8. Critical evaluation and reflection

8.1 Against the requirements

Requirement	Status	Evidence and caveat
O1 / FR5 No double-booking	Met by design; verified by mocks and a manual test	Strong logic evidence; the live-database guarantee depends on the documented atomicity of single-document writes and on the manual test result
O2 Security	Largely met	Controls in Table 7; residual risks in Section 5.5; one defect fix lacks an automated test
O3 Accessibility	Met for tested criteria	Keyboard, labels, focus and non-colour cues tested; no screen-reader or user testing with disabled people
O4 Integration	Met	Adapter, timeout, cache and fallback tested with mocks
O5 Engineering quality	Met	Layered code, 42 tests, CI, repeatable build
FR10 Payments etc.	Not attempted	Out of scope by design

Table 11. Evaluation against objectives.

A26

8.2 What worked and what I would change

The decision that paid off most was resolving the hardest technical question (atomic seat reservation) in week 1 by a small spike, because everything else could then be built on a known foundation. Keeping rules as pure functions made the most important behaviour cheap to test. The decision I would revisit first is the one-document seat model: for larger aircraft or heavy demand on one flight, a separate seat collection with transactions would spread writes and scale better, at the cost of the complexity I avoided. I would also add seat holds, because customers currently lose a seat while typing passenger names, which a timed “held” status would soften.

8.3 Limitations

The main limitations are: the compensation step is not a transaction and does not run if the server crashes mid-way; there are no seat holds; times are stored in UTC and shown in the viewer's time zone, so airport-local times are not modelled; refund tiers are illustrative; tokens cannot be revoked; the rate limiter and cache are per instance; the password policy follows older composition-rule convention that current guidance discourages (Section 5.2); and there are no end-to-end or database-integration tests. Passenger names, although minimal, remain personal data. These limitations bound the claims that can honestly be made for the system.

8.4 Legal, ethical and privacy considerations

Under the General Data Protection Regulation personal data must be processed lawfully, minimised and kept secure (European Parliament and Council of the European Union, 2016). SkyBook collects only a name and email for accounts and passenger names for bookings, holds no payment or passport data, and hashes passwords. A real service would still need a privacy notice, a retention policy and a way for users to export and delete their data, none of which exist here. Ethically, accessibility is a requirement, not a bonus: a seat map that only works with a mouse excludes some travellers, which is why it was designed around keyboard and assistive-technology operation from the start. Since June 2025 the European Accessibility Act has made this a legal requirement for the websites and apps of air passenger transport services in the EU (European Parliament and Council of the European Union, 2019), so the absence of screen-reader testing (Section 8.3) is a compliance risk and not only a quality gap. The refund tiers are described as illustrative so that they are not mistaken for passenger-rights advice (European Parliament and Council of the European Union, 2004). A27

8.5 Future work, prioritised

1. **Update the password policy to current guidance** (minimum 15 characters, no composition rules, blocklist screening; low effort, high security value).
2. **Automated regression test for the passenger-field fix and integration tests against a temporary database** (low effort, high value: closes the gap in Section 6.6).
3. **Seat holds with expiry** (medium effort, high customer value).
4. **Refresh tokens or server-side sessions** to allow revocation (medium effort, high security value).
5. **End-to-end browser tests and screen-reader testing** (medium effort, strengthens accessibility claims).
6. **Seat collection with transactions** if demand or aircraft size grows (high effort, only when needed).

8.6 Reflection, and declaration of AI and external code

I began from the official MERN tutorial (MongoDB, Inc., n.d.) to learn the project layout, then replaced its design with an authenticated, validated, tested and deployed system. I used an AI assistant for two purposes: to explain error messages and to suggest wording for a few explanations; all design decisions, code and test cases were written and checked by me, and I can explain every line. Third-party libraries and services are credited in the references. [Adapt this paragraph to exactly what you used, as the brief requires acknowledgement of any AI-generated content, external code and third-party resources] Working on a concurrency problem taught me to

distrust assumptions: the passenger-field defect and the potentially flaky reference test were both found only because I tested behaviours that the happy path never exercises. A28

9. Conclusion

SkyBook meets its central objective by moving the critical decision (who gets a seat) into one atomic database operation, and surrounds that core with server-side pricing, layered security, an accessible interface, tested business rules and a repeatable deployment. The evidence is strongest for the logic and weakest for behaviour against a real concurrent database and real assistive technology, and the report says so. Future work should first close that evidence gap before adding features.

References

Sources were checked on 4 November 2026. Standards and regulations are cited at the edition current on that date (for example OWASP Top 10:2025, ASVS 5.0.0 and NIST SP 800-63B-4). Online documentation that carries no publication date is cited as (n.d.), with its access date.

Barth, A. (2011) *HTTP state management mechanism*. RFC 6265. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc6265 (Accessed: 4 November 2026).

Bass, L., Clements, P. and Kazman, R. (2021) *Software architecture in practice*. 4th edn. Boston, MA: Addison-Wesley.

Bingler, S., West, M. and Wilander, J. (2025) *Cookies: HTTP state management mechanism*. Internet-Draft draft-ietf-httpbis-rfc6265bis-22 (work in progress). Internet Engineering Task Force. Available at: https://datatracker.ietf.org/doc/draft-ietf-httpbis-rfc6265bis/22/ (Accessed: 4 November 2026).

Bradshaw, S., Brazil, E. and Chodorow, K. (2019) *MongoDB: the definitive guide*. 3rd edn. Sebastopol, CA: O'Reilly Media.

Cohn, M. (2009) *Succeeding with agile: software development using Scrum*. Upper Saddle River, NJ: Addison-Wesley.

European Parliament and Council of the European Union (2004) Regulation (EC) No 261/2004 establishing common rules on compensation and assistance to passengers in the event of denied boarding and of cancellation or long delay of flights. *Official Journal of the European Union*, L 46, pp. 1–8.

European Parliament and Council of the European Union (2016) Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation). *Official Journal of the European Union*, L 119, pp. 1–88.

European Parliament and Council of the European Union (2019) Directive (EU) 2019/882 of 17 April 2019 on the accessibility requirements for products and services. *Official Journal of the European Union*, L 151, pp. 70–115.

express-validator (n.d.) *express-validator documentation*. Available at: https://express-validator.github.io/docs/ (Accessed: 4 November 2026).

Fielding, R.T. (2000) Architectural styles and the design of network-based software architectures. PhD thesis. University of California, Irvine.

Fowler, M. (2002) *Patterns of enterprise application architecture*. Boston, MA: Addison-Wesley.

Fowler, M. (2012) 'TestPyramid', *martinfowler.com*, 1 May. Available at: https://martinfowler.com/bliki/TestPyramid.html (Accessed: 4 November 2026).

Garcia-Molina, H. and Salem, K. (1987) ‘Sagas’, *ACM SIGMOD Record*, 16(3), pp. 249–259.

GitHub (n.d.) *GitHub Actions documentation*. Available at: https://docs.github.com/en/actions (Accessed: 4 November 2026).

Gray, J. (1981) ‘The transaction concept: virtues and limitations’, *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, Cannes, 9–11 September, pp. 144–154.

Haerder, T. and Reuter, A. (1983) ‘Principles of transaction-oriented database recovery’, *ACM Computing Surveys*, 15(4), pp. 287–317.

Jones, M., Bradley, J. and Sakimura, N. (2015) *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc7519 (Accessed: 4 November 2026).

Kleppmann, M. and Riccomini, C. (2026) *Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems*. 2nd edn. Sebastopol, CA: O’Reilly Media.

Martin, R.C. (2008) *Clean code: a handbook of agile software craftsmanship*. Upper Saddle River, NJ: Prentice Hall.

MongoDB, Inc. (n.d.) *How to use MERN stack: a complete guide*. Available at: https://www.mongodb.com/resources/languages/mern-stack-tutorial (Accessed: 4 November 2026).

Mongoose (n.d.) *Mongoose documentation*. Available at: https://mongoosejs.com/docs/ (Accessed: 4 November 2026).

National Institute of Standards and Technology (NIST) (2025) *Digital identity guidelines: authentication and authenticator management*. NIST Special Publication 800-63B-4. Gaithersburg, MD: NIST. doi:10.6028/NIST.SP.800-63b-4.

Nielsen, J. (1994) *10 usability heuristics for user interface design*. Nielsen Norman Group. Available at: https://www.nngroup.com/articles/ten-usability-heuristics/ (Accessed: 4 November 2026).

OpenJS Foundation (n.d.) *Node.js documentation: test runner*. Available at: https://nodejs.org/api/test.html (Accessed: 4 November 2026).

OWASP Foundation (2025a) *OWASP Application Security Verification Standard (ASVS), version 5.0.0*. Available at: https://owasp.org/www-project-application-security-verification-standard/ (Accessed: 4 November 2026).

OWASP Foundation (2025b) *OWASP Top 10:2025*. Available at: https://owasp.org/Top10/2025/ (Accessed: 4 November 2026).

Provos, N. and Mazières, D. (1999) ‘A future-adaptable password scheme’, *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, Monterey, CA, 6–11 June, pp. 81–91.

The React Team (2024) ‘React v19’, *React Blog*, 5 December. Available at: https://react.dev/blog/2024/12/05/react-19 (Accessed: 4 November 2026).

Render (n.d.) *Render documentation*. Available at: https://render.com/docs (Accessed: 4 November 2026).

REST Countries (n.d.) *REST Countries API*. Available at: https://restcountries.com (Accessed: 4 November 2026).

Sheffer, Y., Hardt, D. and Jones, M. (2020) *JSON Web Token best current practices*. RFC 8725. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc8725 (Accessed: 4 November 2026).

Shostack, A. (2014) *Threat modeling: designing for security*. Indianapolis, IN: Wiley.

Sommerville, I. (2016) *Software engineering*. 10th edn. Harlow: Pearson.

Testing Library (n.d.) *Guiding principles*. Available at: https://testing-library.com/docs/guiding-principles/ (Accessed: 4 November 2026).

Todd, W. (2024) ‘Introducing Express v5: a new era for the Node.js framework’, *Express blog*, 15 October. Available at: https://expressjs.com/en/blog/2024-10-15-v5-release (Accessed: 4 November 2026).

W3C (2023) *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. Available at: https://www.w3.org/TR/WCAG22/ (Accessed: 4 November 2026).

A29 A30

Appendix A. API summary

Method	Path	Access	Purpose
POST	/api/auth/register, /login, /logout	Public	Account
GET	/api/airports	Public	Airport list
GET	/api/flights?from=&to=&date=	Public	Search with free-seat counts
GET	/api/flights/:id	Public	Seat map (booking references hidden)
GET	/api/flights/:id/destination	Public	Country facts from the external API
POST	/api/bookings	Login	Reserve seats atomically
GET	/api/bookings, /api/bookings/:ref	Login, owner only	My bookings
DELETE	/api/bookings/:ref	Login, owner only	Cancel with refund
POST	/api/admin/flights	Admin	Create a flight with a seat map
GET	/api/admin/stats	Admin	Totals

Appendix B. Evidence to insert before submission

- [screenshots of the running application: home, seat selection, bookings, administrator page, 390 px view]
- [Atlas screenshots: collections, indexes, a flight with booked seats]
- [terminal output of the 14 server tests and 28 client tests]
- [Lighthouse accessibility report]
- [concurrency test output]
- [CI run and repository commit history]

Marker’s commentary on Report A

Each comment matches a green tag in the report above.

#	Where	Rubric row / brief element	Verdict	Why
A1	Title block	Structure & Presentation; submission requirements	**Meets**	The brief requires a GitHub URL and a deployed URL and says missing or non-working links can lose marks. The block carries both (placeholders mark where a real student inserts working links), plus a word count and a pointer to the AI and external-code declaration. A marker knows immediately that the submission is complete.
A2	Abstract	Structure & Presentation; Research & Analysis	**Meets**	In about 170 words it states the problem, the central design decision, the other contributions, the evidence (42 tests) and that limitations are discussed. It is an honest summary of the report, not advertising. Good abstracts signal critical intent, which the HD descriptor “excellent critical analysis” looks for.
A3	§1.2 Objectives	Research & Analysis (clearly justified requirements)	**Meets**	Five numbered, measurable objectives (O1 correctness to O5 engineering quality). They become the yardstick in Section 8.1, so the final evaluation is checked against what was promised. Weak reports state vague aims (“a secure, easy to use website”) that can never be judged.
A4	§2.1 Research approach	Brief: “critically discuss the research process”; Research & Analysis	**Meets**	The research is described as a process: three strands (user and domain, technical literature, prototyping spikes), each tied to a decision. It cites standards and literature, and it treats the MongoDB tutorial critically (it replaced the tutorial’s unauthenticated, unvalidated design). This is the “strong research and critical evaluation” and “extensive supporting evidence” the upper rubric bands describe.
A5	Table 1 and refund paragraph	Research & Analysis (stakeholder awareness)	**Meets**	Stakeholders include the easily forgotten ones (groups, assistive-technology users, data subjects), and each has a sourced design consequence. The report also correctly separates Regulation 261/2004 (airline-initiated cancellation) from its own passenger-initiated refund tiers: accuracy and restraint, not name-dropping.
A6	Table 2	Part 1: Requirements Analysis and Planning	**Meets**	Each functional requirement has a priority and a testable acceptance criterion. That connects the requirements to the testing section: Section 6 can say which test proves which requirement. A “Won’t” row shows scope control.
A7	Table 3	Part 1: non-functional requirements	**Meets**	Every non-functional requirement has a measurable target and a verification method. Compare the average report’s “secure, fast and easy to use”, which cannot be tested or failed.
A8	§2.5, Table 4	Research & Analysis (feasibility, planning)	**Meets**	A risk register with likelihood, impact and mitigation, plus an iterative plan with MoSCoW and a citation. Feasibility analysis is named explicitly in the HD descriptor for requirements and planning.
A9	§3.1 and Figure 1	Technical Discussion; Brief: “system architecture”	**Meets**	The architecture is shown in a captioned figure and justified with the REST constraints and a quality-attribute trade-off (one service gives same-origin cookies but less flexible scaling). Both benefits and costs are stated. That is what “advanced technical insight and critical evaluation” means.
A10	§3.2	Technical Discussion; Software engineering	**Meets**	The layered structure is named, cited and justified by a concrete benefit (the key rules can be tested without a database). Explaining *why* a structure exists is the step that separates a description from technical discussion.
A11	§3.3 and Table 5	Technical Discussion; Part 1: database design	**Meets**	The most important design decision is compared with its main alternative across explicit criteria, with the cost of the chosen design stated and linked to documentation and transaction theory. The writer avoids the assumption that transactions are always better. This is the clearest single piece of HD-level reasoning in the report.

#	Where	Rubric row / brief element	Verdict	Why
A12	Table 6	Technical Discussion; Brief: “technology choices”	**Meets**	Each choice lists alternatives, a reason and a trade-off, and cites documentation. Weak reports justify with “popular”, “easy” or “the lecturer said”.
A13	§3.5	Part 1: API and web service integration	**Meets**	Explains why the call is made on the server and describes four defensive layers (timeout, adapter, cache, typed error) with the reason for each. This evidences “understanding of asynchronous communication and data handling” and graceful failure.
A14	§3.6	Part 1: Front-end and UX; accessibility	**Meets**	Design decisions are tied to specific WCAG 2.2 success criteria (1.4.1, 2.1.1, 4.1.2, 1.3.1, 3.3.1, 2.4.7) and to usability heuristics. The report evidences “accessibility compliance” instead of just claiming it.
A15	§4.1	Technical Discussion (implementation depth)	**Meets**	Real code is quoted and explained at the level of individual conditions (the conjunction of element matches, array filters, the modified-count check) with a citation for why it is atomic. It also says what the code does *not* promise. This is “strong technical depth and justification”.
A16	§4.2	Technical Discussion; critical evaluation	**Meets**	Names the pattern (saga, with a citation), explains idempotent cancellation, and states the limitation (a crash between the two writes skips the compensation). Naming limits is a feature of top reports.
A17	§4.3	Technical Discussion; Software engineering	**Meets**	Pure functions with an injected clock are linked to testing at exact boundaries and to a clean-code principle. The code excerpt is short and every line is accounted for.
A18	Table 7	Brief: “security implementation”; Part 1: Security	**Meets**	A threat-to-control table mapped to the **current** OWASP Top 10:2025 (published November 2025), produced from a stated threat-modelling pass. It uses the 2025 categories correctly, including the new ones (A03 software supply chain failures, A10 mishandling of exceptional conditions), and shows how weaknesses such as CSRF, SameSite cookies and open redirect now sit under A01. Every control is specific (“rate limit of 20 per 15 minutes”) and the table admits a gap (no alerting, A09). Compare the average report’s feature list, which cites the superseded 2021 edition.
A19	§5.3	Technical Discussion; Part 1: secure session management	**Meets**	Weighs two designs (cookie versus script-readable storage) in terms of the threats each exposes (XSS versus CSRF), says how the chosen design mitigates its own risk, and states the accepted limitation (no revocation). It cites the JWT standard, the JWT best-current-practice RFC and the cookie standards, and is careful to cite the SameSite specification as *work in progress*, which is what it still is. It also records a small improvement (pin the signing algorithm) instead of claiming perfection.
A20	§5.4	Research & Analysis (critical honesty)	**Meets**	Reports a defect found by testing and admits that the fix is not yet covered by an automated test. This is a minor weakness, but disclosing it costs nothing and shows the integrity markers reward. Hiding it would be riskier.
A21	§6.1–6.3, Table 8	Brief: “testing strategy”	**Meets**	Presents a strategy (pyramid, cited), explains why mocks were used, and gives accurate numbers (14 server and 28 client tests, which match the repository). The client tests are justified by a testing principle and doubled as accessibility evidence.
A22	Table 9	Technical Discussion; critical reflection	**Meets**	A defect log with how each defect was found, fixed and what it taught. This turns testing from a claim (“I tested it”) into evidence of learning.

#	Where	Rubric row / brief element	Verdict	Why
A23	§6.6	Technical Discussion (critical evaluation)	**Meets**	States precisely what the mocked tests cannot prove and which piece of evidence (the manual concurrency test) covers the gap. This is the most sophisticated sentence block in the report: it shows the writer understands the limits of their own evidence.
A24	§7	Brief: “deployment process”; Part 1: Cloud deployment	**Meets**	Describes configuration, environment variables, the health check and the continuous-integration link, and a verification checklist with a place for the measured result. Compare the average report’s step list.
A25	Table 10	Part 1: production-ready deployment	**Meets**	A gap analysis between the current deployment and production, including the uncomfortable trade-off (database open to all addresses on the free host) and how it is mitigated. Honest gap analysis earns more than claiming to be “production-ready”.
A26	§8.1, Table 11	Research & Analysis; critical evaluation	**Meets**	Evaluates against the objectives set in Section 1 with a status and a caveat for each, including one “not attempted”. No blanket claim that everything was met.
A27	§8.4	Technical Discussion; legal and ethical	**Meets**	Applies data-protection principles to the actual data held and lists what a real service would still need. It uses the **European Accessibility Act**, applicable since June 2025 to airline websites and apps, to turn the accessibility design from good practice into a compliance matter, and it says that missing screen-reader testing is therefore a compliance risk. It avoids presenting the refund tiers as legal advice. Current law, correctly applied, is a mark of a strong report.
A28	§8.6	Brief: academic integrity (“acknowledged and understood”)	**Meets**	States the starting tutorial, how an AI assistant was used and what stayed the student’s own work, and credits third-party libraries. A placeholder tells the student to adapt the paragraph to what they actually did. A declaration must be true to count.
A29	References	Brief: Harvard in-text citations and a complete list; Structure & Presentation	**Meets**	36 sources, 28 of them with publication dates. They are **current**: OWASP Top 10:2025 and ASVS 5.0.0 (2025), NIST SP 800-63B-4 (2025), the European Accessibility Act (2019, applicable 2025), Express 5 and React 19 release posts (2024), the 2026 second edition of the data-intensive-applications textbook, and the cookie specification cited as a draft because it is still one. They also include classic peer-reviewed papers, books, regulations and RFCs. Every in-text citation has a list entry and every entry is cited (checked automatically); same-author entries are disambiguated (2025a, 2025b). The 8 undated entries are online documentation pages with no publication date, which Harvard cites as (n.d.). Access dates show when this exemplar was prepared; a student records their own. Tutors should still verify each URL.
A30	Whole report and appendices	Structure & Presentation (HD: exceptionally professional)	**Meets**	Numbered sections, captioned tables and figure, consistent terminology, short purposeful code excerpts and appendices (API summary, evidence list). The structure follows the brief’s list of topics in the order a reader needs them.
A31	§5.2	Technical Discussion; critical evaluation; currency of sources	**Meets**	Compares the project’s own password policy with the 2025 NIST guidance (no composition rules, 15-character minimum when a password is the only authenticator, blocklist screening) and concludes that its policy reflects older convention. Criticising your own design against a *current* standard, and ranking the fix as the first future-work item, is the “critical evaluation” and “extensive supporting evidence” the HD descriptors ask for.

Indicative marks for Report A

Rubric row	Band	Indicative %	Reason
Research & Analysis	High Distinction	85–90%	Evidence-led research process, measurable objectives, honest evaluation with caveats
Technical Discussion	High Distinction	85–90%	Trade-offs, code-level explanation, limits of tests stated, security reasoning
Structure & Presentation	High Distinction	88–92%	Professional structure, full consistent referencing, declarations
Overall (Part 2, out of 20)	**High Distinction**	**about 17.5 / 20 (88%)**	What stops it being perfect: the one fix without an automated test (disclosed) and measured results the student must still insert

Report B: average (Merit range)

Why this report is only average, in one paragraph

Report B is complete, readable and accurate about *what* was built. It covers most headings and reaches the length (about 2,059 words). That earns a respectable Pass-to-Merit mark. It stops there because it mostly **describes** and does not **justify, evaluate or evidence**. Technology choices are explained by popularity, security is a list of features with a few wrong terms and big claims, testing lists what was run without a strategy or limits, the conclusion asserts success without evidence, and the references are few, inconsistent and do not match the citations. Each of those is easy to fix (see the “How to improve” column), and fixing them moves a report up a band.

SkyBook Flight Booking System: Technical Report

B9IS130 Web Development for Information Systems

Name: [name] · Student number: [number] · GitHub: [URL] · Live site: [URL] B1

1. Introduction

The aim of this project was to build a flight booking website using the MERN stack. Flight booking is a very popular type of website and millions of people use these websites every day to find and book flights. Because of this it is a good topic for a web development project as it has many different features like searching, booking, user accounts and payments. The website that I built is called SkyBook. B2

The objectives of the project were to create a website where users can search for flights, choose their seats, book the flights and cancel their bookings. I also wanted the website to be secure and easy to use and to be deployed online so that anyone can use it. In this report I will describe the requirements, the technologies I used, the design of the system, the security, the testing and the deployment.

2. Requirements

To find the requirements I looked at some existing flight booking websites. I found that most of them have a search form, a list of results and a seat map where you can pick your seat (W3Schools, 2023). I also looked at the MERN stack tutorial from MongoDB to see how a MERN application is organised (MongoDB, n.d.). B3

The functional requirements are listed below.

- Users can register and log in.
- Users can search for flights by where they are flying from, where they are flying to and the date.
- Users can view a flight and see a seat map.
- Users can choose up to four seats and enter the names of the passengers.
- Users can book the seats and get a booking reference.
- Users can see their bookings and cancel them to get a refund.
- An admin can add flights and see statistics.
- The website shows information about the country the user is flying to.

The non-functional requirements are that the website should be secure, fast, easy to use and work on mobile phones. The users of the website are customers who want to book flights and the admin who manages the flights. B4

3. Technologies Used

The website was built with the MERN stack which is MongoDB, Express, React and Node.js. I chose this stack because the module brief says it is preferred and because it uses JavaScript everywhere which makes it easier to learn. MongoDB is a NoSQL database that stores data as documents. It is faster than SQL databases and is easy to use with JavaScript (MongoDB, n.d.). I used MongoDB Atlas which is the cloud version so that I did not have to install the database on my computer. B5

Express is a framework for Node.js that makes it easier to create web servers and APIs (Express.js, n.d.). I used it to create the REST API for the website. React is a JavaScript library for building user interfaces and it is the most popular front-end library so there are lots of tutorials and help available (React Docs). I used Vite as the build tool because it is fast. For the back end I also used Mongoose to connect to MongoDB, bcrypt to hash the passwords, JSON Web Tokens for logging in, Helmet for security headers and express-validator to check the data that users send.

The website also uses an external API called REST Countries which gives information about countries. I used this to show the capital, languages and currency of the country the user is flying to on the flight page. The data is returned in JSON format.

4. System Design

The system has a client and a server. The client is the React application that the user sees in the browser and the server is the Express application that handles the requests and talks to the database. When the user does something like searching for a flight the React application sends a request to the server, the server gets the data from MongoDB and sends it back as JSON and then React shows it on the page. In production the Express server also serves the React files so there is only one website to deploy. B6

The server code is organised into folders. The routes folder says which URL goes to which function, the controllers folder has the functions that handle the requests, the models folder has the Mongoose schemas and the services folder has the business logic such as calculating the fare and the refund. This makes the code easier to read and to change.

The database has three collections which are users, flights and bookings. A user has a name, email, password hash and role. A flight has a flight number, origin, destination, departure and arrival times, a price and a list of seats. A booking has a reference, the user, the flight, the passengers, the total price and the status. I put the seats inside the flight document so that when two people try to book the same seat only one of them can get it. This is done with one update that only works if all the seats are still free. B7

When the user books seats the server checks that the flight exists and has not left and then it reserves the seats. If the booking cannot be saved then the seats are released again. The price is calculated on the server and not by the browser so that users cannot change it. Rows one and two have extra legroom and cost fifteen euros more. When a user cancels a booking the refund depends on how long before the flight it is. If it is more than 72 hours the refund is 100%, between 24 and 72 hours it is 50% and less than 24 hours there is no refund. Figure 1 shows the code for the refund function. B8

On the client side there are pages for the search, the flight, the bookings, the admin and the login. The seat map is made of buttons and the user can click on a seat to choose it. The website uses a context in React to store who is logged in. There is a footer and a navigation bar on each page and the website is responsive so it works on mobile phones.

The home page has a search form with three fields which are from, to and date. When the user clicks search the website shows a list of the flights for that day. Each flight shows the departure time, the arrival time, how long the flight is, the price and how many seats are left. The user can click select seats to go to the flight page. On the flight page there is a card with information about the country that the user is flying to and a seat map with 48 seats in 8 rows. The seats that are already taken are grey and the seats that the user chooses are blue. Under the seat map there is a form where the user types the name of each passenger and a button to confirm the booking. If the user is not logged in they are sent to the login page first and then they come back to the flight page. After the booking the user goes to the bookings page where there is a message with the booking reference. The bookings page shows the flight, the passengers, the total price and a cancel button. The admin page has some statistics and a form to add a new flight.

5. Security

Security was an important part of the project. Passwords are encrypted using bcrypt before they are saved in the database so nobody can read them. When a user logs in the server creates a JSON Web Token and stores it in a secure cookie so hackers cannot steal it. The cookie is httpOnly and SameSite which means that JavaScript cannot read it. This is more secure than local storage. B9

All the data that users send is validated using express-validator. For example the email must be a real email and the password must have at least 8 characters with a capital letter and a number. SQL injection is prevented because I use Mongoose to talk to the database. The server also uses Helmet which adds security headers to protect against cross site scripting and other attacks (OWASP, 2021). Rate limiting is used on the login and booking routes to stop DDoS attacks.

Some routes are protected and need the user to be logged in. The admin routes also check that the user is an admin. A user can only see their own bookings. Secrets like the database password and the JWT secret are stored in environment variables and not in the code. The admin role cannot be

chosen when registering. It is given by running a script on the server. Because of these features the website is very secure. B10

6. Testing

Testing is an important part of software development (Sommerville, 2016). I wrote automated tests for both the server and the client. There are 14 server tests and 28 client tests and they all pass. The server tests check the seat map, the fare calculation, the booking reference, the refund policy, the routes that need a login, the validation and the booking and cancel functions. The client tests check the API helper, the login, the protected routes, the seat map, the flight page and the format functions. For example one test checks that the seat map shows 24 buttons and another checks that a taken seat is disabled. The tests were run using npm test and they take only a few seconds to run. B11

I also tested the website manually by using all the features. I registered, searched for flights, booked seats, cancelled bookings and used the admin page. I also tried it on my phone. I did not find any problems in the final version of the website so I think the testing was successful. During development I found a few small bugs such as a button that did not look right and I fixed them. B12

7. Deployment

The website was deployed to Render which is a cloud hosting platform. First I created a free cluster on MongoDB Atlas and a database user. Then I allowed access from all IP addresses so that Render could connect. I pushed my code to a GitHub repository and then I created a new web service on Render and connected it to the repository. I set the build command to npm run build and the start command to npm start. I also added the environment variables for the database connection string, the JWT secret and set NODE_ENV to production. B13

After Render built the project the website was available online. I tested it by registering a new user and booking a flight and everything worked. The free version of Render goes to sleep when nobody uses it so the first request can take a while. The link to the website is at the top of this report and the code is on GitHub.

8. Challenges

During the project I had some challenges. The first challenge was connecting to the database because the connection string needs the right password and the right IP access in Atlas. The second challenge was making sure that two users cannot book the same seat, which I solved by putting the seats inside the flight document. The third challenge was learning how to deploy to Render because I had never deployed a website before. I solved these challenges by reading the documentation and by asking questions in class. I also had to learn how to use Git and GitHub properly and I made commits regularly while I was building the website.

9. Conclusion

In conclusion the project was successful and I managed to build a flight booking website with all the features that I planned. The website is secure, fast and easy to use and it has been deployed online. I learned a lot about the MERN stack, about security and about testing and deployment. In the future I could add more features such as online payments, email confirmations and the ability to

change a booking. Overall I am happy with the project and I think it meets the requirements of the brief. B14

References

Express.js (n.d.) *Express*. https://expressjs.com

MongoDB (n.d.) *MERN stack tutorial*. www.mongodb.com

React Docs. https://react.dev

Sommerville, I. *Software Engineering*. Pearson.

W3Schools (2023) *Node.js tutorial*. https://www.w3schools.com/nodejs/

B15

Marker’s commentary on Report B

Each comment matches an amber or red tag in the report above. The last column says what would earn the marks.

#	Where	Rubric row / brief element	Verdict	Why	How to improve
B1	Title block	Structure & Presentation; submission requirements	Partly	Name and both links are present, which is good. Missing: a date, a word count, and any declaration about AI or external code, which the brief asks students to acknowledge.	Add a short declaration section stating exactly which AI tools, tutorials and external code were used and for what.
B2	§1	Research & Analysis (problem definition)	Partly	Reads as general background. “Millions of people use these websites” is unsupported; the problem the project actually solves (concurrent seats, money, personal data) is never identified, and the objectives cannot be measured.	Define the problem precisely, add measurable objectives, support claims with sources.
B3	§2, first paragraph	Research & Analysis; Brief: research process	Partly	Says existing sites were examined but names none and draws no conclusions. The only source cited is W3Schools, a general tutorial site, for a claim about booking websites, which it does not support. The MongoDB tutorial is cited without a year and is described, not critiqued.	Describe how research was done, cite standards and academic or official sources, and say how each source changed the design.
B4	§2 requirements	Part 1: Requirements; non-functional requirements	Partly	The functional list is reasonable and matches the application. Non-functional requirements (“secure, fast, easy to use”) are untestable. There are no priorities, stakeholder analysis, acceptance criteria, risks or feasibility discussion.	Give each requirement a priority and acceptance criterion; give each non-functional requirement a measurable target and a test.

#	Where	Rubric row / brief element	Verdict	Why	How to improve
B5	§3 technologies	Brief: technology choices; Technical Discussion	**Partly**	Reasons are “preferred in the brief”, “easy to learn”, “popular”, “fast”. “MongoDB is faster than SQL databases” is an unsupported claim attributed to a tutorial. No alternatives are compared and no trade-offs are stated. Descriptions of what each tool is replace justification.	Compare at least one alternative per decision against stated criteria; replace opinions with cited evidence or measurements.
B6	§4 first paragraph	Brief: system architecture	**Partly**	Accurate description of client, server and database, and of serving the client from Express. But there is no diagram, no naming of the architectural style, and no discussion of why this arrangement was chosen over others.	Add a labelled architecture figure and explain the trade-offs of the choices.
B7	§4 database paragraph	Part 1: database design; Technical Discussion	**Partly**	The key idea (seats inside the flight so only one booking can win) is correct, which is a real strength. It is not compared with an alternative, not linked to any source and its cost (contention) is not mentioned.	Compare with a separate seat collection and transactions; cite documentation; state the cost.
B8	§4 booking and refund paragraph	Technical Discussion (implementation depth)	**Partly**	Lists correct mechanisms (rollback, server-side price, refund tiers) but only says *what* happens. “Figure 1 shows the code” refers to a figure that does not exist, and no code is analysed. Reads as a feature list.	Quote short code excerpts and explain line by line why it is written that way and what it cannot guarantee.
B9	§5 first paragraph	Brief: security implementation; accuracy	**Fails**	“Encrypted using bcrypt” is wrong: passwords are *hashed*, and the difference matters. “Stored in a secure cookie so hackers cannot steal it” is an overclaim: an httpOnly cookie resists script theft but creates request-forgery risk that needs a separate defence. The comparison with local storage is right in direction but incomplete.	Use precise terms (hash, salt, cost factor), explain the cookie trade-off (XSS versus CSRF) and state the limits.
B10	§5 remaining paragraphs	Brief: security; Part 1: Security	**Partly**	Describes real controls (validation, Helmet, rate limiting, role checks, environment variables) but as a list with no threat model. “SQL injection is prevented” misnames the risk for a MongoDB application (query-operator injection). Rate limiting is said to stop DDoS attacks, which it does not. “Very secure” is an unsupported conclusion. OWASP is cited in the text as 2021, but that edition has been superseded by the OWASP Top 10:2025, and the source is missing from the reference list anyway. The password rules are described as good practice without noting that current guidance has moved away from them.	Map threats to controls against the current OWASP Top 10:2025, correct the terminology, state residual risks and check current guidance (for example on passwords).

#	Where	Rubric row / brief element	Verdict	Why	How to improve
B11	§6 first paragraph	Brief: testing strategy	**Partly**	Accurate counts (14 server and 28 client tests) and a sensible description of what they cover. But there is no strategy (why these layers, why mocks), no results table, no boundary-testing discussion and no limits. The citation to Sommerville supports a generic statement and does not match the incomplete list entry.	Explain the test layers, why mocks were used, what the tests cannot prove, and show results in a table.
B12	§6 manual testing	Research & Analysis (critical evaluation)	**Partly**	“I did not find any problems” is not credible and has no evidence. No security attack tests are described. The only defects mentioned are trivial. Contrast with a defect log that shows what testing taught.	Describe specific manual and attack tests with results, and keep a log of defects found and fixed.
B13	§7	Brief: deployment process	**Partly**	A clear, accurate sequence of steps, which is the part many students do well. Missing: why the database was opened to all addresses and the risk of that, verification evidence, the CI pipeline, and what production would need beyond a free host.	Add a configuration rationale, a verification checklist with results, and a gap analysis.
B14	§9 Conclusion	Research & Analysis; critical evaluation	**Partly**	Claims the project “meets all requirements” and is “secure, fast and easy to use” with no evidence. No limitations, no evaluation against objectives, and future work is a wish list without priorities.	Evaluate against the stated objectives with evidence, list real limitations and prioritise future work.
B15	References	Brief: Harvard in-text and list; Structure & Presentation	**Fails**	Only five entries, in an inconsistent format: missing years (React Docs, Sommerville), no edition, a bare web address with no path (MongoDB), no access dates. In-text citations do not match the list: (Sommerville, 2016) and (OWASP, 2021) appear in the text but the list holds an undated Sommerville entry and no OWASP entry. A tutorial site is used where an official or academic source was needed. The brief says inadequate referencing can be penalised.	Use one Harvard style consistently; cite official documentation, standards and academic sources; make every in-text citation match a complete list entry and vice versa.

Indicative marks for Report B

Rubric row	Band	Indicative %	Reason
Research & Analysis	Pass to Merit	48–55%	Basic research (one weak source for a general claim), vague objectives, no critical evaluation, conclusion asserts success
Technical Discussion	Merit	52–58%	Clear and accurate explanations of the implementation, but justification is thin and security has errors
Structure & Presentation	Merit	50–58%	Clear headings and readable; paragraphs long; no figures, captions or tables; reference list inconsistent and mismatched

Rubric row	Band	Indicative %	Reason
Overall (Part 2, out of 20)	**Merit (average)**	**about 11 / 20 (55%)**	Competent description, little analysis or evidence

Report C: Fail

Why this report fails, in one paragraph

Report C does not meet the brief. It is about 636 words against a 2,000 minimum, has no structure, and omits whole required topics (security, testing, deployment are a sentence each, and research, architecture and evaluation are effectively absent). It cites no sources at all, contains a paragraph that looks copied, admits using code the writer does not understand, and claims features (payments, email, an AI recommender) that the project does not have. It gives no URLs. Each of those alone would lose marks; together they put it in the Fail band on all three rubric rows, and the unacknowledged code and false claims could trigger an academic-integrity referral in addition to a low mark.

flight booking project

name: [name] C1

For my project i made a flight booking website. I used the MERN stack. It was very hard but i learned a lot from it and i spent a lot of time on it. First i watched a tutorial on youtube and followed it and then i changed the colors and the names so it would be a flight booking website and not what the tutorial was. I also got some code from the internet because i did not know how to do the seats and some of it i dont really understand but it works. C2

The website has a home page where you can search for flights and then you can see the flights and book them and pay with Stripe and you get an email confirmation after you book. There is also an AI that recommends the cheapest flights for you and a chat bot. There is also an admin who can do everything on the website. You can also see your bookings and change your seat after you book. C3

The technologies that i used are react for the front end and node for the back end and mongo for the database and express is also used. React is a javascript library. I chose these because the lecturer said to use them and because they are popular. MongoDB is a document-oriented NoSQL database that stores data in flexible JSON-like documents, which allows schemas to evolve without costly migrations and supports horizontal scaling through sharding, making it ideal for modern applications with rapidly changing requirements. C4

The requirements of the website are that users can book flights. The website is for people who want to fly. I did not do a lot of research because i did not have time but i looked at some websites like ryanair. C5

For the security the passwords are safe. I used a token for the login and the website is very secure and cannot be hacked. There is a login page and a register page. If you are not logged in you cant book. C6

I tested the website and it works. There were some bugs but i fixed them. I tested it on my laptop and my phone. I did not write tests because i did not know how to do them but i clicked on everything. C7

The website is on the internet. It is on GitHub. The link is in the other file i uploaded. It is on a free server so it can be slow sometimes. I had a problem with the database connection at the start because the password had a special character but i fixed it by changing the password. C8

The design of the website is that it has a navigation bar at the top and the pages are in the middle. The seats are in a grid and you click them and they go blue. I used a colour scheme with blue and white. The bookings page shows your bookings. The code is organised in folders for the client and the server. C9

There were a lot of problems when i did the project. The biggest problem was that i started late and there was not enough time. Also the tutorial was for a different type of website so i had to change a lot of things. I also had problems with git because i pushed the wrong files once. In the end i got it working. C10

In the future i want to add more stuff like more airports and a better design and maybe a mobile app. I think my project is good and should get a good mark because i worked very hard on it and spent many hours on it and the website works and looks nice. C11

I think i have met all the requirements of the brief. C12

Marker’s commentary on Report C

Each comment matches a red tag in the report above.

#	Where	Rubric row / brief element	Verdict	Why	How to improve
C1	Title block	Structure & Presentation; submission requirements	**Fails**	Only a name. No module, student number, GitHub URL or deployed URL. The brief says failure to provide working links may negatively impact marks.	Add identification, both links and a word count.
C2	Opening paragraph	Brief: academic integrity (“acknowledged and understood”)	**Fails**	Admits following a tutorial and “changing the colours and names”, and using code from the internet that the writer “does not really understand”. External code and its source are not acknowledged, and understanding is explicitly absent. This is a serious academic-integrity risk, and the report offers no reference or declaration.	Credit every tutorial and external source, say what was changed, and only submit code you can explain line by line.
C3	Second paragraph	Accuracy; academic integrity	**Fails**	Claims Stripe payments, email confirmations, an AI recommender, a chatbot and seat changes. The project simulates payment and has none of the others. Describing features that do not exist misrepresents the work and will be discovered when the marker opens the repository.	Describe only what exists and can be demonstrated; list the rest as future work.

#	Where	Rubric row / brief element	Verdict	Why	How to improve
C4	Technology paragraph	Technical Discussion; Brief: technology choices; referencing	Fails	Technologies are named, not justified (“the lecturer said to”). The last sentence is written in a very different, polished voice from the rest of the report and carries no citation, which suggests it was copied. Uncited copied text is plagiarism.	Justify each choice against alternatives in the student’s own words and cite sources.
C5	Requirements paragraph	Research & Analysis (Fail: limited or no research)	Fails	No requirements, no users analysed, and the writer says little research was done. Mentioning an airline name is not analysis.	Add stakeholders, functional and non-functional requirements and sources.
C6	Security paragraph	Brief: security implementation	Fails	Assertions without explanation: “the passwords are safe”, “cannot be hacked”. No hashing, token, validation or authorisation is described, and “cannot be hacked” is never true. No threats are named.	Describe each control, the threat it addresses, its limits and how it was tested.
C7	Testing paragraph	Brief: testing strategy	Fails	No tests were written and no strategy exists; “clicked on everything” is not testing evidence. “Some bugs but I fixed them” gives no detail.	Add automated tests and a testing section with strategy, results and a defect log.
C8	Deployment paragraph	Brief: deployment process; working links	Fails	The site is said to be “on the internet” but then “on GitHub”, which is not a deployment, and the link is “in the other file”. There is no platform, configuration or process. The brief requires a deployed URL in the submission.	State the platform, configuration and steps, and give the working URL in the report and the submission.
C9	Design paragraph	Brief: system architecture; Technical Discussion	Fails	Describes colours and layout only. No architecture, data model, API or explanation of how the parts fit together.	Add an architecture figure, a data model and explanations of key design decisions.
C10	Problems paragraph	Research & Analysis (critical evaluation)	Fails	A narrative about starting late and Git mistakes. Process difficulties are not analysed and there are no lessons about the system itself.	Evaluate the system against objectives and reflect on specific technical decisions.
C11	Final paragraph	Research & Analysis; critical evaluation	Fails	Argues for a mark on the basis of effort. Marks reward evidenced achievement against the criteria. No limitations or future work beyond a wish list.	Evaluate with evidence; prioritise future work.
C12	Last line, and the whole document	Brief: Harvard referencing; Structure & Presentation	Fails	Claims all requirements are met when almost none is evidenced. There are no in-text citations and no reference list at all, despite the brief requiring Harvard referencing for tutorials, libraries and APIs. The report is about 640 words, well under the 2,000 minimum, has no headings and is written informally.	Meet the minimum length with substance, structure the report, and cite and list every source.

Indicative marks for Report C

Rubric row	Band	Indicative %	Reason
Research & Analysis	Fail	10–15%	“Limited or no research”: no requirements analysis, no sources, no evaluation

Rubric row	Band	Indicative %	Reason
Technical Discussion	Fail	8–12%	“Poor explanations”: technologies named, nothing explained; false and unsupported claims
Structure & Presentation	Fail	10–15%	“Poorly structured with major errors”: no headings, informal, under the minimum length, no references
Overall (Part 2, out of 20)	**Fail**	**about 2.5 / 20 (12%)**	Plus a likely academic-integrity concern

Side by side: what separates the three

Aspect	Report C (Fail)	Report B (average)	Report A (High Distinction)
Problem and objectives	None	General aim, no measures	Defined problem; five measurable objectives used later as the yardstick
Research	“Did not do a lot”	Looked at some websites; one weak source	Three strands, spikes, standards and literature, each tied to a decision
Requirements	None	List of features; vague qualities	Prioritised, with acceptance criteria and measurable targets; risk register
Technology choices	“The lecturer said”	“Popular”, “easy”, “fast”	Alternatives, criteria, trade-offs, cited
Architecture and data	Colours	Accurate description	Figure, style named, decision compared with an alternative, cost admitted
Security	“Cannot be hacked”	Feature list with wrong terms	Threat-to-control table, trade-offs, residual risk
Testing	None	Counts and coverage list	Strategy, results, defect log, limits of the evidence
Deployment	No URL	Steps	Configuration, CI, verification, production gap analysis
Evaluation	Asks for marks for effort	“Meets all requirements”	Objective-by-objective evaluation with caveats; prioritised future work
References	None	Five, inconsistent, mismatched	36 current sources, consistent, every citation matched
Integrity	Undeclared, not understood, false claims	Declaration missing	Specific, honest declaration
Typical mistake to avoid	Not engaging with the brief	Describing instead of evaluating	Over-claiming; the exemplar deliberately states its limits

The ladder: what moves a report up a band

To move from…	Do these things
Fail → Pass	Reach the minimum length with real content; add a heading for every topic in the brief; write one paragraph each on security, testing and deployment that says *what* and *how*; cite and list every source you use; give both URLs; describe only what exists; declare external code and AI use
Pass → Merit	Justify each choice with a reason beyond “popular”; add an architecture figure; explain code, not just features; correct terminology (hash, not encrypt); use at least ten proper sources in one consistent Harvard style; make citations and list match

To move from…	Do these things
Merit → Distinction	Compare alternatives with trade-offs; map threats to controls; present a test strategy with results; add measurable requirements and a risk register; evaluate against objectives with evidence; state limitations
Distinction → High Distinction	Explain what your own evidence cannot prove and how you covered the gap; log defects and what they taught; analyse the production gap honestly; cite academic and standards sources, not only documentation; polish structure, captions and consistency; make every claim proportionate to its evidence

Guidance for tutors and students

Using the exemplars in teaching

- **Blind-marking exercise.** Give students Reports B and C *without* the commentary and ask them to mark them against the rubric and the brief's checklist, then compare with the commentary.
- **Find the problems.** Ask students to spot the three wrong or over-claimed statements in B's security section (encrypted instead of hashed, "hackers cannot steal it", "stops DDoS") before showing the comments.
- **Reference audit.** Ask students to match every in-text citation in B to its list entry, then do the same for A, and note why A's match.
- **Rewrite a paragraph.** Pick one paragraph of B (for example the technology paragraph) and have students rewrite it to Distinction level using alternatives and cited evidence.

Common pitfalls these samples illustrate

- Describing *what* the system does and stopping, instead of explaining *why* and *how well*.
- Claiming a quality ("secure", "scalable", "tested") without a threat, a test or a number behind it.
- Using tutorial sites as sources for claims they do not support.
- Reference lists that do not match the in-text citations.
- Declaring nothing about AI or external code, or declaring it vaguely.
- Writing about features that are not in the repository.

Before you submit your own report

- Is every item in the brief's list (research process, architecture, technology choices, security, testing, deployment) a clearly labelled section?
- Does every claim have evidence: a source, a test result, a number or a screenshot?
- Does every in-text citation appear in the reference list, and every list entry in the text, in one consistent Harvard style?
- Have you stated your limitations, and what your evidence cannot show?
- Have you declared tutorials, external code and AI use truthfully and specifically, and can you explain every line you submit?
- Do both the repository URL and the deployed URL work from a different computer?
- Are you at or above the word minimum with substance, not padding?

Sample reports prepared for B9IS130 as marking exemplars. The system described is the SkyBook (MERN) sample project. Marks are indicative, assume equal weighting of the three Part 2 rubric rows, and are not an official moderation. Verify all references and web addresses before reuse.